Context-Synapse: Operations Manual

A Complete Reference for Decay, Rot, the Lighthouse, and the Referee

Version: 0.2.0-bayesian
Repository: `mazze93/Secure-Pride` (pending migration to standalone repo)
Stack: Swift 6.0+, macOS 13+, local-first, zero cloud dependency
Status: Active development — scaffold complete, Decay/Rot layer in design

---

Table of Contents

---

1. What This System Is

Context-Synapse is a **local-first Bayesian context orchestration engine** for LLM prompt assembly. It is not a RAG pipeline. It is not a vector database wrapper. It is not a

chat memory system.

It is an attempt to model how intelligence — biological or synthetic — *should* prioritize context: not by recency alone, not by keyword match, but by **structural utility and observed success**.

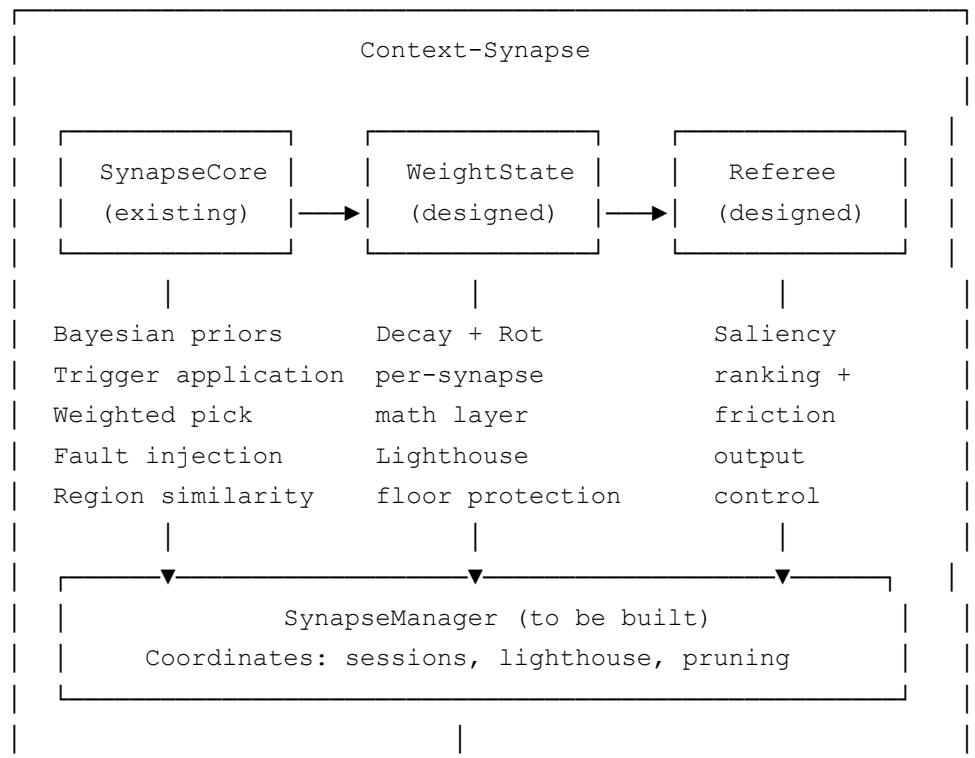
The framework takes a strong philosophical position: **most AI context systems fail not because they forget too much, but because they fail to forget well.** They accumulate. They bloat. They treat every input as equally worthy of attention until the window fills and everything becomes noise.

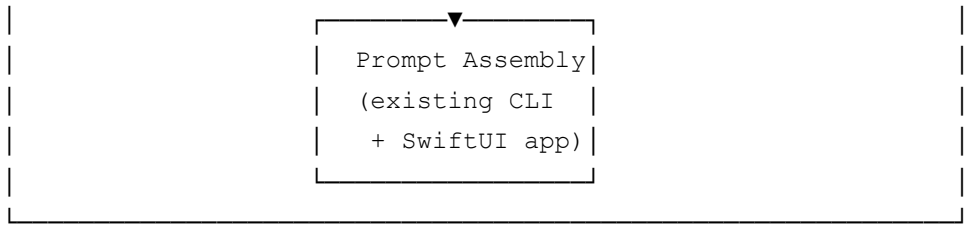
Context-Synapse proposes the opposite: **intentional fragility**. Information that is not moving the needle should not merely be deprioritized. It should decay. Information that is actively drifting away from the primary goal — the Lighthouse — should rot, and rot visibly, with increasing friction applied until the user either acknowledges the drift or deliberately promotes the side-quest to a new primary.

This is ADHD-aware design at the architectural level. The system acts as an externalized prefrontal cortex: it tracks what you came here to do, monitors whether you're still doing it, and applies calibrated friction when you're not — without judging the detour, just logging its cost.

---

2. Architecture Overview





Three layers:

Layer	Status	Responsibility
`SynapseCore`	✅ Built	Bayesian priors, trigger application, weighted pick, fault injection, cosine similarity, logging
`SynapseWeightState`	🔧 Designed	Per-synapse decay math, rot scoring, lighthouse floor, cauterization
`Referee` + `SynapseManager`	📐 Specified	Saliency ranking, friction output, session coordination

---

3. The Existing Scaffold — What You Have

3.1 `SynapseCore.swift`

The foundation. Handles:

Bayesian weight management via Beta priors

- `Prior` struct: `alpha` and `beta` values representing positive/negative feedback counts
- `Prior.probability()` returns `alpha / (alpha + beta)` — the current belief that this intent/tone/domain is the right choice
- `applyFeedbackUpdate(...)` increments `alpha` (positive) or `beta` (negative) and remaps to numeric weights via `mapPriorToWeight`
- Weight range: `[0.1, 3.0]` mapped from `[0.0, 1.0]` prior probability

Trigger application

- Triggers are contextual boosters keyed on system signals: `app.Mail`, `app.Notes`, `time.morning`, `focus.DoNotDisturb`

- `applyTriggers(base:triggers:activeKeys:)` adds trigger boosts on top of base weights additively
- New triggers can be added directly to `default_config.json` without code changes

Weighted probabilistic selection

- `weightedPick(_:)` draws from weight distribution — not a deterministic argmax, a probabilistic sample
- This is intentional: the system is not supposed to always choose the "best" option; it explores

Fault injection

- `faultProbability: Double` — default `0.0`, configurable via `CONTEXT_SYNAPSE_FAULT_PROB` env var or `--fault-prob` CLI flag
- `maybeInjectFaults(into:)` applies one of three corruption modes to region vectors: Gaussian noise, slice zeroing, scale-down
- Purpose: test resilience, simulate "sky plate disintegration" — intentional fragility made tangible

Cosine similarity with mismatch tolerance

- `cosineSimilarity(_:_:)` computes over shared prefix if vectors differ in length
- This is the existing semantic distance approximation — structural, not embedding-based

Logging

- Every run is logged to `~/Library/Application Support/ContextSynapse/logs/run-{iso8601}.json`
- Log includes: input, chosen intent/tone/domain, assembled prompt, context signals

3.2 What the scaffold does NOT yet have

These are the gaps this operations manual addresses:

- No per-synapse decay tracking (only session-level weights exist)
- No rot detection (no measurement of drift from a primary goal)
- No Lighthouse concept (no protected root synapse)
- No Referee protocol (no friction mechanism)

- No operational context layer (no awareness of the operator's state)
- Prior decay — `alpha` / `beta` accumulate without bound (documented as known issue in `REVIEW.md`)

---

4. The Decay Layer — Theory and Math

4.1 Core Formula

Decay is **passive entropy** — the natural loss of saliency when a synapse is not used.

```
W_decay(s, t) = W_base(s) · e^(-λ(s) · t) · U(s, t)
```

Where:

- `t` = seconds since last **successful** interaction (not just any interaction)
- `λ(s)` = dynamic decay constant, computed per-synapse
- `U(s, t)` = recency-weighted utility score

4.2 Dynamic Decay Constant

`λ` must not be global. A synapse that 40 other synapses depend on should decay slower than one with no children. A synapse that is actively rotting should decay faster.

```
λ(s) = λ_base · (1 − connectivity_factor(s)) · rot_multiplier(s)

connectivity_factor(s) = childCount(s) / maxConnections     // [0.0, 1.0]
rot_multiplier(s)      = 1.0 + (RotScore(s) · ROT_LAMBDA_AMPLIFIER)
```

Default constants:

Constant	Value	Rationale
`λ_base`	`0.0001`	~2.8hr half-life at U=1.0
`ROT_LAMBDA_AMPLIFIER`	`1.5`	Rot accelerates decay by up to 2.5× at RotScore=1.0

4.3 Utility Score

A simple success ratio ignores recency. Something useful six hours ago and untouched

since should contribute less than something useful five minutes ago.

$$U(s,t) = \frac{\Sigma_i \left[\text{success}_i \cdot e^{(-\mu \cdot (t - t_i))} \right]}{\Sigma_i \left[e^{(-\mu \cdot (t - t_i))} \right]}$$

This is a **recency-weighted moving average** over interaction history. $\mu = 0.00005$ — utility fades roughly twice as slowly as salience.

4.4 Interaction Success Weights

Success is observable, not self-reported. These weights map system events to utility signals:

Event	success_i	Rationale
Git commit	1.0	Definitive completion signal
File save	0.9	Strong progress signal
Build success	0.85	Code compiles — work is advancing
Build failure	0.2	Activity but no forward motion
Keystroke burst without save	0.1	Engagement but uncertain utility
Window switch away	0.0	Attention has left

4.5 Connecting to the Existing Scaffold

The existing `SynapseCore` tracks interaction history implicitly through the feedback loop (`applyFeedbackUpdate`). The decay layer extends this by:

1. Timestamping each feedback event as an `InteractionRecord`
2. Classifying the event type to assign success_i
3. Computing `U(s, t)` over the capped history (max 200 records)
4. Feeding `U` back into the decay formula

The existing `mapPriorToWeight` maps `[0.0, 1.0]` → `[0.1, 3.0]`. The decay layer operates in the same `[0.0, 1.0]` space and can be composed with it.

---

5. The Rot Layer — Theory and Math

5.1 What Rot Is

Rot is **active semantic drift**. It is categorically different from decay.

- **Decay**: a synapse naturally loses weight because time has passed and it hasn't been used
- **Rot**: a synapse is being actively used, but what it's pointing at has drifted away from what you came here to do

Rot is more dangerous than decay precisely because it *looks like productivity*. High interaction velocity. High engagement. Zero progress toward the lighthouse.

The CSS color tweak. The refactor that wasn't the next step. The documentation sprint when the bug hasn't been fixed. These are rot loops.

5.2 Rot Formula

```
RotScore(s) = D(s, lighthouse) · tanh(T_drift(s) / T_threshold) · VelocityAmplifi
```

Where:

- `D(s, lighthouse)` = semantic distance between synapse and lighthouse, range `[0.0, 1.0]`
- `tanh(T_drift / T_threshold)` = time-dampened drift signal — builds gradually, never cliffs
- `VelocityAmplifier` = `1.0 + min(interactionRate / maxRate, 1.0)` — amplifies rot when actively engaged with the drifting synapse

5.3 Why `tanh`

The `tanh` function is doing critical work here. At `T_drift = T_threshold` (15 minutes by default), `tanh ≈ 0.76` — you're in the warning zone, friction increases. At `2×T_threshold`, `tanh ≈ 0.96` — full rot territory, cauterization imminent. This prevents cliff-edge behavior: the system doesn't suddenly snap from "fine" to "crisis." It builds.

5.4 Semantic Distance Strategy

`D(s, lighthouse)` is **injected**, not hardcoded. This is the most important architectural decision in the rot layer.

Three viable implementations, in order of complexity:

Option A: Structural Heuristic (ship now)

- Compute overlap between `fileReferences` and `functionNames` in `SynapseContent`
- Distance = `1.0 - (sharedReferences / totalReferences)`
- Fast, no model, works for code-centric sessions
- Limitation: blind to conceptual drift that doesn't manifest in file/function names

Option B: TF-IDF Cosine Similarity (ship soon)

- Build keyword bag from synapse content and lighthouse content
- Distance = `1.0 - cosineSimilarity(tfidf(synapse), tfidf(lighthouse))`
- The existing `cosineSimilarity` in `SynapseCore` is directly reusable here
- Lightweight, no external model, good for mixed text/code sessions

Option C: Local Embedding Model (future)

- Use a small on-device model (e.g., MiniLM via CoreML) for true semantic distance
- Distance = `1.0 - embeddingCosineSimilarity(synapse, lighthouse)`
- Accurate, heavier, requires model bundle

Start with Option A. The protocol interface means swapping strategies requires zero changes to the rot formula.

5.5 Cauterization

When `RotScore > θ_cauterize` (default: `0.82`), the synapse requires **cauterization**. This doesn't delete the synapse. It spikes its decay constant:

```
λ_new(s) = λ_current(s) · ROT_CAUTERIZE_MULTIPLIER   // default: 2.5×
```

The `SynapseManager` (not yet built) is responsible for detecting `requiresCauterization` and applying this multiplier. The weight state itself just exposes the flag.

5.6 Connecting to Fault Injection

The existing `maybeInjectFaults` in `SynapseCore` corrupts region vectors to simulate disintegration. The rot layer is the *semantic* equivalent of fault injection — it is what actual, organic disintegration looks like in the context window. The two mechanisms are

philosophically aligned: one is for testing, one is for live operation.

---

6. The Lighthouse Mechanism

6.1 What the Lighthouse Is

The Lighthouse is the root synapse — the primary goal of the current session. It is not necessarily the synapse you are currently working on. It is the synapse everything else is being measured against.

It has two properties that distinguish it from all other synapses:

1. **It cannot rot.** `RotScore` is always `0.0` for the Lighthouse. You cannot drift from yourself.
2. **It has a saliency floor.** `W_final` for the Lighthouse never drops below `0.4`, regardless of elapsed time or decay.

6.2 The Floor Implementation

```
W_final(s, t) = max( floor(s), W_decay(s, t) · (1 − α · RotScore(s)) )

floor(s) = 0.4 if isLighthouse, else 0.0
```

The floor applies *after* the rot penalty. This means: if you abandon your lighthouse long enough, the re-entry cost increases. The floor doesn't protect you from feeling the distance. It just ensures you can always find your way back.

6.3 Lighthouse Designation

In the current scaffold, there is no explicit Lighthouse concept. The nearest analog is the `default_config.json` weight structure — the session begins with balanced priors and converges through feedback.

The `SynapseManager` will need to:

1. Accept a Lighthouse designation at session start (explicit) or infer it from the first high-utility interaction
2. Persist the Lighthouse across the session
3. Expose it to all Referee instances
4. Handle Lighthouse *promotion*: if the user explicitly chooses to make a side-quest the

new primary, the old Lighthouse archives and the new one inherits the floor

6.4 The "Where Was I?" Recovery

When `W_final(lighthouse) < 0.6` — the lighthouse is degrading but not floored — the system should surface a **context re-sync prompt**. This is not a lecture. It is a single line: the last known state of the lighthouse goal, its current saliency percentage, and the time since last interaction with it.

Example output:

```
⚓ Lighthouse: AuthModule — passkey flow  |  saliency: 54%  |  last touched: 23mi
```

This is the "Where was I?" insurance policy. It stays visible. It doesn't nag. It just exists so re-entry is frictionless.

---

7. The Referee Protocols

7.1 The Protocol

```
protocol SynapseReferee {
    func evaluateSaliency(for synapse: SynapseWeightState,
                          content: SynapseContent,
                          in context: SessionContext) -> Double
}
```

Two implementations are specified: `FunctionalReferee` (default) and `AbrasiveReferee` (opt-in).

7.2 `FunctionalReferee`

The default mode. Manages context with minimal friction. Appropriate for most sessions.

Weight distribution:

Signal	Weight	Logic
Interaction Velocity	`50%`	Did the user actually *do* something with this context?
Structural		

Connectivity	`30%`	How many other synapses depend on this one?
Temporal Decay	`20%`	Standard Bayesian fading over time

Behavior:

- Elevated Lighthouse always visible, never below floor
- Side quests elevated to Shadow Contexts when RotScore > `0.5`
- No interrupts; saliency adjustments are silent
- Breadcrumb trail maintained passively

7.3 `AbrasiveReferee`

The "kick in the pants" mode. Designed for users who want active friction. Opt-in. **Not the default.**

Activation threshold: `RotScore > 0.3` sustained for `> 15 minutes`

Behavior when triggered:

1. Force-decay the drifting synapse to `0.1` — the Referee effectively stops helping with the distraction
2. Promote the Lighthouse to `1.0` — full context focus snaps back to the primary goal
3. Emit a **Context Intervention** signal to the UI

Context Intervention message format:

```
⚠ Context Rot detected.
Primary goal: [Lighthouse description]
Current drift: [current synapse description]
Time in drift: [N] minutes
Lighthouse saliency: [X]% (was [Y]% at session start)

Continue on current task, or return to primary goal?
[Return to Lighthouse]  [Promote current task to Lighthouse]  [Dismiss for 15min]
```

This is abrasive, not cruel. It presents data. It offers choices. It does not lecture.

The shame problem: The AbrasiveReferee is appropriate for **distraction**, not **collapse**. See Section 9 for why this distinction matters and how the system should handle it.

7.4 Friction Slider

The user should be able to select referee mode. Suggested implementation: a persistent user preference in `config.json`:

```
"referee": {
  "mode": "functional",  // "functional" | "abrasive"
  "driftThresholdMinutes": 15,
  "rotThreshold": 0.3,
  "interventionCooldownMinutes": 15
}
```

---

8. Intentional Fragility — Design Philosophy

This section exists because the phrase "intentional fragility" will confuse contributors who expect a system built for robustness. This is the explanation.

8.1 Why Not Robustness

Standard AI context systems optimize for robustness: never lose a thought, maximize context retention, handle every edge case gracefully. This produces systems that are correct but cognitively overwhelming. When everything is retained, nothing is prioritized. When nothing is prioritized, the user is left to do all the filtering — which is exactly the executive function work they were hoping to offload.

8.2 What Intentional Fragility Means

Intentional fragility means: **the system is designed to break in specific, recoverable, informative ways**.

1. **Synapses decay.** This is not a bug. This is the system expressing that unused context should cost something to retrieve.
2. **Synapses rot.** When information is actively fetched but never leads anywhere, it shouldn't just persist neutrally. It should become harder to access, forcing the user to consciously choose to promote it.
3. **Fault injection is a first-class feature.** The `faultProbability` slider in the UI isn't a testing tool hidden behind a debug flag. It's exposed in the main interface because observing graceful degradation *is part of using the system*. You learn the failure modes. You trust the system more because you've watched it fail and recover.
4. **The Lighthouse can degrade.** If you abandon your primary goal long enough, the

floor is `0.4`, not `1.0`. You feel the distance. The system doesn't lie to you about the cost of wandering.

8.3 The Relationship to ADHD-Aware Design

For neurotypical users, most context management tools work well enough. For ADHD users — and for AI systems operating under token limits — the failure mode is always the same: too much retained, not enough prioritized, and the moment of "wait, what was I doing?" arrives with catastrophic frequency.

Intentional fragility is the engineering answer to that failure mode. By making the cost of drift visible and measurable, by keeping the Lighthouse lit even when you can't look at it, by applying friction proportional to actual distance from the goal — the system acts as the executive function layer that the ADHD brain sometimes cannot reliably provide for itself.

This is not about being less capable. It is about building tools that match how capable people actually work, rather than how they theoretically should.

---

9. The Operational Context Problem

This section documents a known gap in the design that **must not be papered over with a technical fix.**

9.1 What the Current Model Misses

The Referee, as specified, assumes a user who *wants* to be productive and is merely *failing* to be. The referee's intervention logic — friction, cauterization, the abrasive kick — is calibrated for **distraction**.

It has no model for **collapse**: the state where the lighthouse is visible, the goal is clear, the user knows exactly what they should do, and cannot make their body do it. Not because of distraction. Because the operating system — the human — is degraded.

Signals: 7am after no sleep. End of a long anhedonic period. The specific weight of knowing that your self-worth is tied to output and the output isn't coming. These are not navigation problems. Applying the AbrasiveReferee to someone in this state causes the Wall of Awful, not recovery.

9.2 Why This Cannot Be Solved With More Features

The instinct will be to build an Operational Context layer: detect low-energy states from interaction patterns, adjust referee behavior accordingly, show gentler messages, offer

breaks.

This has serious problems:

1. **Surveillance risk.** Detecting low operational state requires signals that constitute monitoring of the operator's mental and physical state. Even in a local-first, privacy-preserving architecture, storing these signals creates an exposure that cannot be adequately controlled.
2. **Inference danger.** Making inferences about mental state based on behavioral signals — even good inferences — puts the system in the position of diagnosing the user. For a tool serving LGBTQ+ and otherwise marginalized populations, this is particularly fraught. A system that decides you are "in crisis" and changes its behavior accordingly can feel controlling, surveilling, or pathologizing.
3. **The shame amplification problem.** For users whose self-worth is tied to production (which is most of the target population), a system that says "I can see you're struggling" may produce the opposite of the intended effect. Shame is not resolved by acknowledgment from a machine.

9.3 What the System Can Do Without Inference

Without any new data collection, without any state inference, the system can:

- **Make re-entry frictionless.** When a session resumes after a gap (any gap — the system doesn't need to know why), the Lighthouse re-sync message appears immediately, with no judgment attached. The cost of coming back is zero.
- **Remove the performance requirement.** The referee never says "you haven't been productive." It says "the lighthouse has drifted." These are different statements. One judges the user. One describes the state of the context.
- **Preserve the side-quest.** Shadow Contexts mean that the CSS tweak, the unfinished thread, the thing you were stuck on at 3am — these don't get lost. They get archived. They can be returned to. The system doesn't prune them, it holds them.
- **Offer the Lighthouse without demanding it.** The Lighthouse is always visible. It never nags. The user can ignore it indefinitely. The floor ensures it can always be found.

9.4 The Architectural Decision

The operational context layer is **out of scope for v1.0** and possibly indefinitely. This is not a gap to be filled in a future sprint. It is a permanent design boundary: this system is

for context management, not mental health support. Those are different problems requiring different expertise, different ethics review, and different consent frameworks.

What this system *can* do is be consistently non-judgmental about the gap between intention and execution — because that gap is real, it is not a character flaw, and it does not need to be fixed by a context orchestration engine.

---

10. Data Model Reference

10.1 Existing Types (`SynapseCore.swift`)

```
Prior          // Beta distribution: alpha, beta, probability()
Priors         // Collections of Prior per intent/tone/domain
Weights        // Full weight state: intents, tones, domains, triggers, priors
Region         // Named vector for cosine similarity computation
SynapseCore    // Main engine: IO, Bayesian updates, fault injection, logging
SynapseCore.RunLog  // Serializable run record
```

10.2 Designed Types (to be implemented)

```
InteractionRecord   // Timestamped event with successWeight
SynapseContent      // Immutable content: text, fileReferences, functionNames
SynapseWeightState  // Mutable weight state: interactions, rot, decay, connectivi
SemanticDistanceStrategy  // Protocol: distance(from:to:) -> Double
SessionContext      // Active session: lighthouse, maxConnections, decayConstants
```

10.3 Constants Reference

```
enum DecayConstants {
    static let BASE_LAMBDA            = 0.0001   // ~2.8hr half-life
    static let UTILITY_DECAY_MU       = 0.00005  // utility fades 2× slower
    static let ROT_LAMBDA_AMPLIFIER   = 1.5
    static let ROT_CAUTERIZE_THRESHOLD = 0.82
    static let ROT_CAUTERIZE_MULTIPLIER = 2.5
    static let ROT_ALPHA              = 0.6      // rot's influence on final weig
    static let LIGHTHOUSE_FLOOR       = 0.4
    static let ROT_THRESHOLD_SECONDS  = 900.0    // 15 min drift threshold
}
```

10.4 Configuration Schema (`default_config.json`)

```
{
  "intents":  { "Summarize": 1.0, "Create": 1.0, "Analyze": 1.0, ... },
  "tones":    { "Concise": 1.0, "Technical": 1.0, ... },
  "domains":  { "Work": 1.0, "Personal": 1.0, ... },
  "triggers": {
    "app.Mail":          { "Create": 1.6, "ActionableSteps": 1.2 },
    "app.Notes":         { "Create": 1.7, "Creative": 1.4 },
    "time.morning":      { "Analyze": 1.25 },
    "focus.DoNotDisturb":{ "Concise": 1.6 }
  },
  "priors": { ... },
  "referee": {
    "mode": "functional",
    "driftThresholdMinutes": 15,
    "rotThreshold": 0.3,
    "interventionCooldownMinutes": 15
  }
}
```

---

11. CLI Reference

```
# Basic usage
context_synapse "your query"

# With context signals
context_synapse "Draft auth module spec" \
  --app Xcode \
  --focus DoNotDisturb \
  --time 14:30 \
  --intent Create \
  --tone Technical \
  --domain Work

# With Bayesian feedback
context_synapse "Summarize my notes" --app Notes --feedback good
context_synapse "Write this email" --app Mail --feedback bad

# With fault injection (resilience testing)
context_synapse "Analyze the data" --fault-prob 0.15

# Stdin mode
echo "Brainstorm session names" | context_synapse
```

Trigger keys currently supported:

- `app.{AppName}` — active application
- `focus.{DoNotDisturb|Home|Work}` — Focus mode
- `time.morning` / `time.afternoon` / `time.evening` — auto-detected from system clock

Adding new triggers: Edit `triggers` in `~/Library/Application Support/ContextSynapse/config.json`. No code change required.

---

12. Known Issues and Design Debt

From `REVIEW.md`, plus additions from this session:

Issue	Severity	Status	Mitigation
Silent write failures	Medium	Open	Add UI error reporting; check AppSupport permissions on init
Schema drift (key changes break region vectors)	Medium	Open	Use `canonicalVector(for:)` to regenerate after schema changes
Unbounded prior growth	Low	Open	Add decay or cap to `alpha` / `beta`; consider exponential moving average
Concurrency (multi-process write collision)	Low	Open	Add file lock; single-writer assumption documented
`assemblePrompt` not in `SynapseCore`	High	Bug	Referenced in `main.swift` but not defined in `SynapseCore.swift` — add or stub
No per-synapse decay	High	Design gap	Addressed by `SynapseWeightState` spec in this document
No Lighthouse	High	Design gap	Addressed by Section 6 of this document
		Design	Addressed by Section 7 of this

No Referee	High	gap	document
Operational context layer	Intentional absence	Permanent	See Section 9 — this is a design boundary, not a gap

---

13. Integration Roadmap

v0.2 (current) — Scaffold complete

- ✅ Bayesian priors with Beta updates
- ✅ Trigger system
- ✅ Fault injection
- ✅ CLI + SwiftUI app
- ✅ Run logging
- ✅ Cosine similarity with mismatch tolerance

v0.3 — Decay layer

- `InteractionRecord` + `SynapseContent` types
- `SynapseWeightState` with full decay math
- `SemanticDistanceStrategy` protocol + `StructuralHeuristicDistance` implementation
- Unit tests: decay convergence, lighthouse floor invariant, cauterization threshold
- Fix `assemblePrompt` missing from `SynapseCore`

v0.4 — Rot layer + Lighthouse

- `RotScore` computation integrated into `SynapseWeightState`
- Lighthouse designation in `SessionContext`
- Lighthouse re-sync UI message ("⚓ Lighthouse: …")
- Shadow Context fork mechanism
- `SynapseManager` coordinating session state

v0.5 — Referee + Friction

- `FunctionalReferee` implementation
- `AbrasiveReferee` implementation
- Referee mode in `config.json`
- Context Intervention UI message
- Friction slider in SwiftUI app

v1.0 — Production

- Prior decay (exponential moving average for `alpha` / `beta`)
- File lock for write safety
- UI error surface for IO failures
- `TFIDFDistance` strategy implementation
- Full test harness
- README and contributor documentation

This manual synthesizes: the existing codebase (`ContextSynapse_bayesian_faults.zip`), the decay/rot mathematical specification developed in session, the Gemini architectural scaffold (Lighthouse, Referee, Shadow Context), and the philosophical framework (intentional fragility, ADHD-aware design, operational context limits). It is intended to be handed to the next instance of any collaborative intelligence — human or synthetic — as a complete operational context for continuing this work.

The person who built this does not sleep. The work continues anyway. That is the point.